OSInsight-AI

Advanced Disk Scheduling Simulator with Explainable AI | Project Report

Project type	Operating Systems simulation
Core algorithms	FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK
AI capability	Explainable multi-objective policy selection
Technology	Python, CustomTkinter, ReportLab, pytest

Abstract

OSInsight-AI is an advanced disk scheduling simulator designed for operating-systems education, laboratory demonstrations, and performance analysis. The project models how a traditional disk controller chooses the next cylinder to serve when requests arrive in a queue. Rather than presenting one algorithm as universally best, it executes six classic policies on the same workload and exposes their trade-offs through consistent metrics. The simulator includes FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK. Its central enhancement is an explainable AI policy selector that evaluates queue locality, seek movement, estimated waiting time, throughput, and fairness before recommending a policy for a user-selected objective. The result is a practical learning tool that combines established operating-systems concepts with transparent AI-assisted decision making. It runs locally, accepts custom queues, generates realistic mixed workloads, provides an interactive desktop interface, and exports a compact report suitable for submission records.

1. Problem Statement and Motivation

Disk scheduling remains a foundational topic because storage requests compete for a shared service mechanism. In rotating disks, poor ordering creates unnecessary head movement, longer response times, and uneven service. Even though solid-state drives do not have a physical arm, the underlying problem of scheduling queued work is still relevant to operating systems, database engines, cloud services, and networked storage. Many classroom examples show only the final head movement of one policy. Students therefore find it difficult to compare policies fairly, understand starvation risks, or explain why a policy fits one workload but not another. OSInsight-AI addresses this gap by allowing a single workload to be evaluated across multiple policies at once. It turns a static numerical exercise into a repeatable experiment with visible paths, quantitative comparison, and a reasoned recommendation.

2. Project Objectives

The project has five objectives. First, it must accurately simulate the core disk scheduling algorithms under configurable disk geometry, head position, request queue, and sweep direction. Second, it must present useful metrics rather than only a service order. Third, it must support exploration by generating workloads that resemble a mixture of local interactive I/O and occasional distant batch requests. Fourth, it must use AI in a responsible and understandable way: the recommendation should be reproducible from clear features and weights, not hidden behind an unsupported claim. Finally, it must remain clean and practical for a college project. The application has a focused Python structure, readable domain models, automated tests, a desktop dashboard, and a PDF export that can be used during a viva or laboratory assessment.

3. System Architecture

The architecture is intentionally compact. The scheduling domain contains immutable models for disk requests, workloads, and results. A workload validates disk bounds, initial head location, direction, and request values before any simulation begins. The algorithms module then produces a service order and a movement path for the chosen policy. The metrics layer derives total seek, average seek, estimated average wait, throughput, and Jain's fairness index. The AI advisor receives the same validated workload, runs every eligible algorithm, normalizes their outcomes, and ranks them according to the selected objective. The graphical dashboard is responsible only for input, presentation, workload generation, and report export. This separation makes the algorithms easy to test and prevents user-interface code from hiding scheduling logic.

4. Scheduling Algorithms

First Come First Served preserves arrival order. It is simple and fair in an arrival-order sense, but it may move the head back and forth across the disk. Shortest Seek Time First repeatedly selects the nearest pending cylinder. It often minimizes immediate movement but can delay distant requests when nearby traffic continues to arrive. SCAN moves in one direction until a disk boundary, serving requests on the way, then reverses. C-SCAN services in one direction and returns to the opposite boundary, producing more uniform wait behavior for requests at different positions. LOOK and C-LOOK apply the same ideas but turn around at the last pending request rather than always travelling to a physical disk edge. These policies demonstrate the key balance between efficiency, predictability, and fairness.

5. Performance Metrics

Total seek is the total number of cylinders travelled by the head and is the main efficiency measure. Average seek divides that movement by the number of serviced requests so queues of different sizes can be compared. Average wait is an abstract latency estimate: for each request, the simulator measures how far the head has travelled before reaching that request and averages the values. Throughput is expressed as requests serviced per movement unit; it is a useful comparative signal rather than a real device benchmark. Fairness uses Jain's fairness index on estimated waits. A value closer to one indicates that requests receive more even service. Together, these measurements prevent a misleading conclusion based only on the lowest total seek. A highly efficient policy may still have a poor fairness or latency outcome for a particular queue.

6. Explainable AI Recommendation

The AI component is an explainable multi-objective policy selector. It first profiles the distribution of request cylinders using the mean and population spread. A tight spread indicates high locality, while a broad spread represents a dispersed workload. The advisor then simulates all six algorithms and converts seek cost and wait estimate into normalized efficiency and latency values. The user chooses one of three objectives: Performance gives the greatest weight to movement efficiency, Fairness emphasizes Jain's fairness score, and Balanced distributes importance across efficiency, fairness, and latency. The score is calculated from visible weighted components. The advisor reports the winning algorithm, a confidence estimate based on the score margin and queue size, and a plain-language explanation. This design is deliberately transparent, so a student can defend the recommendation and reproduce it using the displayed table.

7. User Experience and Workflow

The CustomTkinter dashboard opens with a clear workload panel and a comparison workspace. A user enters the initial head, the number of disk cylinders, and a comma-separated request queue. For directional algorithms, the user chooses upward or downward sweep. Pressing Run AI Recommendation validates the values, evaluates the policies, and updates the results table. The table marks the selected policy and displays total seek, average seek, fairness, average wait, and throughput for every algorithm. The selected service path is shown directly below the

table. The Generate Realistic Workload action creates a cluster around a random center with several long-distance outliers. This provides a better demonstration than a purely uniform random queue because it shows how locality affects scheduling. The Export Simulation Report action preserves the current analysis as a PDF.

8. Implementation Quality

The code follows a human-readable structure. DiskRequest, Workload, and ScheduleResult are frozen dataclasses, which makes simulation state predictable and prevents accidental modification. Workload validation catches invalid geometry, an out-of-range head, an empty queue, and invalid direction early. Algorithm implementations are deterministic, so the same input always produces the same result. The sweep implementation distinguishes boundary-based SCAN variants from LOOK variants, and circular variants include the return movement required by their policy. The calculation of estimated waits follows the actual path, including sweep boundaries and circular wraps. The AI selector is separate from the algorithm code and never changes a result; it only compares outcomes. This separation supports maintainability and makes unit testing straightforward.

9. Testing and Validation

Automated tests validate three important contracts. The first test runs every implemented algorithm and confirms that every request in the workload is serviced exactly once. The second uses a standard mixed queue to verify that SSTF improves on FCFS for that known case. The third confirms that the AI advisor returns a valid algorithm, a bounded confidence score, and an explanation containing fairness information. Manual validation complements the tests: users can enter edge-oriented requests, reverse the sweep direction, compare generated workloads, inspect the service path, and export a report. The application also validates numeric input before a run and communicates errors through a clear dialog. These checks are appropriate for a simulation project because correctness depends on both algorithm behavior and understandable presentation.

9.1 Experimental Methodology

A useful experiment changes one variable at a time. For example, a student can hold the queue fixed and run it with the head near the centre, near cylinder zero, and near the maximum cylinder. The same queue can then be evaluated in both sweep directions. Another experiment compares a concentrated queue against a dispersed queue of equal length. The concentrated case typically rewards local movement, whereas the dispersed case can reveal the regularity provided by circular policies. The result table records all relevant values together, so students should state both the best total seek result and the fairness consequence. The AI objective can also be varied while the workload remains fixed. If the selected algorithm changes between Performance and Fairness modes, the explanation demonstrates that the system is responding to an explicit priority rather than producing a random recommendation. This method supports reproducible observations and encourages evidence-based conclusions.

9.2 Reliability, Privacy, and Responsible Use

OSInsight-AI is local-first: request queues are entered in the desktop application and are not transmitted to a remote service. The AI advisor does not call an external model, collect telemetry, or require an account. This is appropriate for an academic tool because its conclusions must be inspectable and repeatable. Input validation rejects values outside the declared disk range and prevents invalid geometry from reaching the scheduling functions. The report export includes only the selected simulation details, not information about the user's computer. Responsible AI in this project means avoiding a misleading black-box claim. The recommendation is an assistance mechanism, not an authority that overrides engineering judgement. A user can inspect every candidate result, choose a different policy, and explain why a different operating environment could justify a different trade-off. This approach preserves user control and reinforces the analytical purpose of the simulator.

9.3 Interpretation of Results

Results must be interpreted in context. A lower total seek is normally desirable because it represents less head movement, but it is not the only definition of quality. In an interactive system, a policy with moderately higher movement may be preferable if it avoids leaving requests at one end of the disk behind. Conversely, a batch-oriented workload may reasonably prioritize aggregate throughput. SCAN-family algorithms often provide predictable movement patterns, while LOOK-family algorithms avoid unnecessary travel when no request exists at an edge. SSTF can be particularly strong for a static clustered queue but requires careful discussion of starvation under continuing arrivals. FCFS remains valuable as a baseline because it is simple and preserves order. The AI result should therefore be presented as a ranked decision for the simulator's workload model and chosen objective, followed by the metrics that justify it. This is a more rigorous conclusion than naming a universally best algorithm.

10. Limitations and Future Scope

The simulator intentionally models logical cylinders and relative movement rather than a manufacturer-specific storage device. It does not represent rotational latency, transfer size, cache effects, command merging, real arrival times, or SSD firmware internals. The average wait and throughput values are comparative abstractions, not millisecond measurements. Future versions could support timed arrivals, request priorities, deadlines, queue visualizations, live animation, CSV import, benchmark datasets, and a learned policy trained on labelled workload classes. A future AI extension should remain explainable by displaying features, training data boundaries, and evaluation quality. The present design establishes a strong baseline because the student can verify each policy before introducing more complex prediction models.

11. Conclusion

OSInsight-AI delivers a complete advanced disk scheduling project with a clear academic purpose. It moves beyond isolated hand calculations by providing six algorithms, configurable experiments, realistic queue generation, interpretable metrics, and an explainable AI recommendation. The dashboard makes comparison immediate, while the modular code makes the logic easy to inspect and test. Most importantly, the AI feature supports learning rather than obscuring it: it states what it observed in the workload, how objectives alter the ranking, and why one policy was selected. The project is therefore suitable for a final submission, live demonstration, and further research into adaptive operating-systems scheduling.

Report prepared for the OSInsight-AI Advanced Disk Scheduling Simulator project.